Name: _____

Class: _____

Mr. Rawson • GCSE Computer Science Year 11 2026

Algorithms 1 — Computational Thinking — ANSWERS

You already do this

VOCAB **Algorithm** (OCR 2.1.1) — a set of clear steps, in order, that solves a problem. A program is an algorithm written in a programming language

Over half term you wrote six programs. Before writing any of them, you worked out what the program had to do, what it could ignore, and the steps in order. That thinking has a name: **computational thinking**. OCR names three parts of it, and this sheet gives each one its name, using your own holiday programs as the examples.

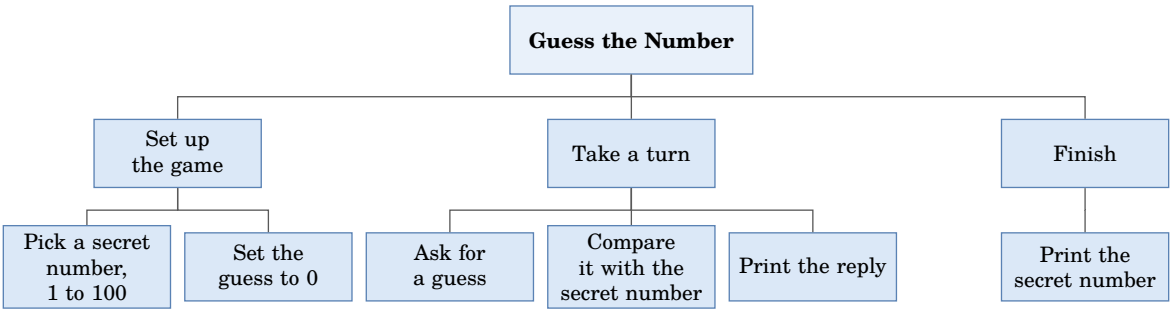
Principle	What it means
Decomposition	Breaking a big problem into smaller problems, each easier to solve.
Abstraction	Keeping only the details that matter, and leaving the rest out.
Algorithmic thinking	Working out the steps, in order, that solve the problem.

1 Decomposition

VOCAB **Decomposition** (OCR 2.1.1) — breaking a problem down into smaller problems, each of which does one job and is easier to solve

The rule. Write the whole problem in one box at the top. Underneath, write the two to four smaller problems it is made of. Keep breaking each one down until every bottom box is a single job you could write as one procedure. This drawing is a **structure diagram**.

Worked example: Guess the Number.



Read it from the top down. The top box is the whole game. Each box underneath is one part of it, and each bottom box is one job: `Ask for a guess` is a single `input()` line in your program.

In Programming 8 you met the same idea in code: each bottom box could become one sub program, and the main program calls them in order. Breaking a program into sub programs **is** decomposition.

Why decompose? These are the answers the exam wants.

Each small problem is easier to solve than the whole problem.

Each part can be written and tested on its own.

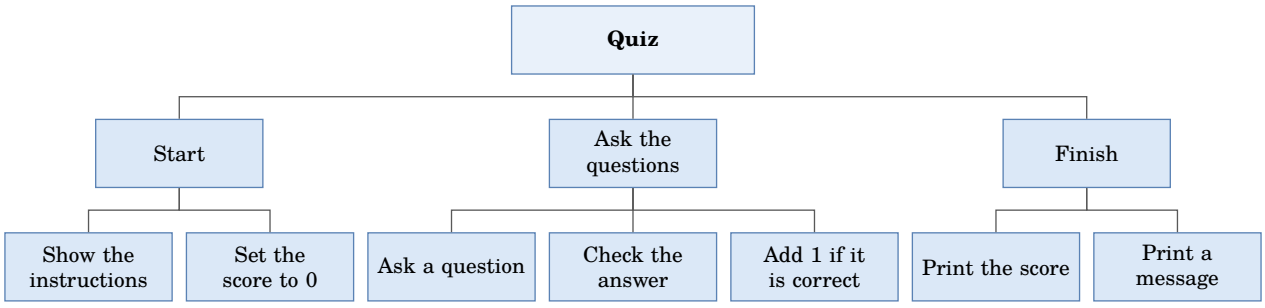
Parts can be reused, in this program or in another one.

Different programmers can work on different parts at the same time.

Now try • Draw a structure diagram for a **quiz program**. It shows some instructions, asks the user five questions, adds 1 to their score for each correct answer, and at the end prints the score with a message.

Check • The whole problem is in one box at the top, there are three or four boxes on the level below, and every bottom box is one job you could write as one procedure.

ANSWER



Other wording is fine. What earns the mark is the shape: whole problem at the top, each level a breakdown of the box above it.

2 Abstraction

> **VOCAB** **Abstraction** (OCR 2.1.1) — removing the details that do not matter, so that only the information needed to solve the problem is left

The rule. List the details of the problem. For each one, ask: **if this changed, would the answer change?** If yes, keep it. If no, leave it out.

Worked example: the London Underground map. The map is drawn to answer one question: *which trains do I take to get from here to there?*

Kept, because they change your route	Left out, because they do not
The stations, in order	The real distances between stations
Which line each station is on	The streets and buildings above ground
Where you can change lines	The real shape and direction of the tunnels

A map with every street on it would be harder to use, not easier. Leaving details out is what makes it useful.

Your holiday program did the same. Guess the Number keeps two pieces of data: the **secret number** and the **guess**. It leaves out the player's name, how long they take and how many turns they have had, because none of those changes whether the guess is too high, too low or correct.

Now try • A taxi app works out the fare for a journey. Sort these six details into the two columns: **distance travelled** • **driver's name** • **time spent waiting in traffic** • **colour of the car** • **day or night rate** • **radio station playing**.

Check • Three details in each column. Test each one with the rule: every detail you kept changes the fare.

Keep: it changes the fare	Leave out: it does not
Distance travelled	Driver's name
Time spent waiting in traffic	Colour of the car
Day or night rate	Radio station playing

3 Algorithmic thinking

VOCAB **Algorithmic thinking** (OCR 2.1.1) — working out the steps needed to solve a problem, in the right order, including where to make a decision and where to repeat

The rule. Before you write any code, write the steps as a numbered list in plain English. Say where the program makes a decision (**if**) and what it repeats (**while**).

Worked example: Sum of Positive Numbers, from your holiday sheet.

1. Set the total to 0.
2. Ask the user for a number.
3. While the number is 0 or more: add it to the total, then ask for the next number.
4. Print the total.

Step 3 is the repeat, and the words *while the number is 0 or more* are its condition: that is the `while` loop in your program. Someone who has never written code could follow these four steps with a pencil and get 17 for the inputs 5, 12, −1.

Now try • Write the steps of your **Even or Odd** holiday program as a numbered list in plain English.

Check • Three or four steps, with one decision, and each step says exactly what happens. Someone who has never coded could follow them.

ANSWER

1. Ask the user for a number.
2. Divide the number by 2 and find the remainder.
3. If the remainder is 0, print `Even`.
4. Otherwise, print `Odd`.

4 Inputs, processes and outputs

VOCAB **Input, process, output** (OCR 2.1.2) — the data that goes into a program, what the program does with it, and what the program gives back

The rule. **Inputs** are the data the program is given: here, what the user types. **Processes** are what the program does with the data: calculations, comparisons, repeats. **Outputs** are what the program shows the user.

Worked example: Sum of Positive Numbers.

Inputs	Processes	Outputs
The numbers the user types, one at a time	Add each number to the running total	The total, in a sentence
A negative number, to stop	Check whether the number is 0 or more	

Now try • Fill in the table for your **Guess the Number** holiday program.

Check • The secret number goes under **Processes**: the program makes it, and the user never types it. Inputs hold only what the user types.

Inputs	Processes	Outputs

Each guess the user types	Pick a random secret number, 1 to 100.
	Compare the guess with the secret number.
	Repeat until the guess is correct.
		`Too high`
		`Too low`
		`Correct! The number was` and the number

Stretch Noughts and crosses

You met a noughts and crosses board in Programming 7, as a 2D array. Draw a structure diagram for the whole game: two players take turns to place X or O on a 3×3 board until one of them has three in a line, or the board is full.

Check • The top box is the whole game, there are at least three levels, and the bottom boxes include each of the three ways to win: a row, a column and a diagonal.

ANSWER

One good answer, written as a list (each indent is one level down the diagram):

- **Noughts and crosses**
  - **Set up**: draw an empty board • X goes first
  - **Take a turn**: show the board • ask for a row and a column • check the square is empty • place the mark
  - **Check the result**: check each row • each column • each diagonal • check whether the board is full
  - **Switch player**
  - **Finish**: print the winner, or a draw

The exam answers, one line each

Asked for	Write
Decomposition	Breaking a problem down into smaller problems that are easier to solve.
Abstraction	Removing unnecessary detail, so only the information needed is kept.
Algorithmic thinking	Working out the steps, in order, that solve the problem.
Structure diagram	A diagram that shows a problem decomposed from the top down.
Inputs, processes, outputs	What goes in, what the program does with it, what comes out.